Python for Data Analysts

Learning Roadmap + Practice Question Workbook

How to use this workbook: Learn each subtopic, then answer the 10 questions without looking at notes. For coding questions, write and run the code yourself. Mark questions you cannot solve and revisit them after practice.

Roadmap

- 1. Python Fundamentals
- 2. Python Data Structures
- 3. NumPy
- 4. Pandas
- 5. Data Visualization
- 6. Exploratory Data Analysis (EDA)
- 7. Real Data Analyst Projects
- 8. Interview Preparation

1. Python Fundamentals

Variables

1. Define a variable in Python and explain how assignment works.
2. Create variables for a customer's name, age, and total purchase amount.
3. What is the difference between assigning a value and comparing two values?
4. Can a Python variable change from an integer to a string? Demonstrate.
5. Create three variables in one line and assign them different values.
6. What naming rules should you follow when creating Python variables?
7. Identify the problem with the variable name `customer-name`.
8. Swap the values of two variables without using a third variable.
9. Use a variable to calculate the total price of 5 products costing 120 each.
10. Why are meaningful variable names important in data analysis?

Data Types

1. What are the main built-in Python data types used in beginner data analysis?
2. How is an integer different from a floating-point number?
3. Determine the data type of `25`, `25.0`, `"25"`, and `True`.
4. Convert the string `"150"` into a numeric value.
5. Convert the number 42 into a string.
6. When would a Boolean value be useful in a data-analysis program?
7. What happens if you try to add an integer and a string?
8. Explain why data types matter when working with datasets.
9. Create variables representing product name, quantity, price, and whether it is active.
10. Write a small example containing at least four different Python data types.

Strings

1. Create a string containing the text `Data Analyst`.
2. How do you find the length of a string?
3. Extract the first five characters from `Python`.
4. Convert `data analyst` to uppercase and lowercase.
5. Remove leading and trailing spaces from a customer name.
6. Concatenate a first name and last name into one string.
7. Use an f-string to display a customer's name and total sales.
8. Check whether the word `SQL` appears inside a sentence.
9. Replace `Mumbai` with `Delhi` in a string.
10. Why is string cleaning important in real-world data analysis?

Numbers & Arithmetic

1. Calculate the sum, difference, product, and quotient of two numbers.
2. What is the difference between `/`, `//`, and `%` in Python?
3. Calculate a 10% discount on a price of 2,500.
4. Calculate the percentage growth from 800 to 1,000.
5. Calculate the average of five sales values.
6. Use exponentiation to calculate 2 to the power of 5.
7. Round 87.4567 to two decimal places.
8. Explain why floating-point calculations can sometimes produce unexpected results.
9. Calculate revenue from quantity and unit price.
10. Create a simple profit calculation using revenue and cost.

Booleans & Comparisons

1. What values can a Boolean variable contain?
2. Write an expression that checks whether sales are greater than 10,000.
3. Compare two customer ages using `==`.
4. Explain the difference between `==` and `!=`.
5. Write a condition that checks whether a score is between 50 and 100.
6. Use `and` to check two conditions simultaneously.
7. Use `or` to check whether a customer is from either of two regions.
8. Use `not` to reverse a Boolean condition.
9. Create a Boolean variable named `is_returning_customer`.
10. Why are Boolean expressions important for filtering data?

if / elif / else

1. Write an `if` statement that prints a message when sales exceed 10,000.
2. Create an `if/else` condition that labels a customer as adult or minor.
3. Use `elif` to classify scores as low, medium, or high.
4. Write a condition that applies free shipping when an order exceeds 2,000.
5. Create a simple sales-performance classification with three categories.
6. What happens when none of the conditions in an if/elif/else chain is true?
7. Write a nested conditional and explain why nested logic can become difficult to maintain.
8. How would you handle a missing value using conditional logic?
9. Create a condition that identifies unusually large transactions.
10. Why is conditional logic useful during data cleaning?

for Loops

1. Write a loop that prints the numbers 1 through 10.
2. Loop through a list of five sales values and print each value.
3. Calculate the total of a list of sales using a `for` loop.
4. Count how many values in a list are greater than 1,000.
5. Loop through customer names and print a greeting for each.
6. Use `range()` to generate values from 0 through 20.
7. Explain the difference between iterating over values and indexes.
8. Use a loop to create a list of squared numbers.
9. Find the largest value in a list using a loop.
10. Why are loops useful, and when might a vectorized Pandas operation be preferable?

while Loops

1. Write a while loop that counts from 1 to 5.
2. Create a loop that continues until a sales target is reached.
3. What condition must eventually become false in a while loop?
4. Explain how an infinite loop can happen.
5. Use a while loop to process values until a zero is encountered.
6. Write a loop that repeatedly asks for a value until a valid positive number is entered.
7. Compare a while loop with a for loop.
8. Why should while loops be used carefully in data-processing scripts?
9. Create a while loop that doubles a number until it exceeds 1,000.
10. Give one data-analysis situation where a while loop could be appropriate.

Functions

1. Define a Python function that adds two numbers.
2. What are parameters and arguments?
3. Create a function that calculates revenue from quantity and price.
4. Create a function that returns the average of a list of numbers.
5. Why should a function return a value rather than only print it?
6. Create a function that labels a transaction as high or low value.
7. Explain local versus global variables at a beginner level.
8. Give a function a default parameter value.
9. How can functions reduce duplicated analysis code?
10. Write a function that cleans a customer name by stripping spaces and converting it to title case.

Error Handling

1. What is an exception in Python?
2. Why can `try`/`except` be useful when reading or processing data?
3. Write a simple example that handles invalid numeric input.
4. What is the purpose of `finally`?
5. What is the difference between a syntax error and a runtime exception?
6. Handle a possible division-by-zero error.
7. Why should you avoid using a completely empty `except` block in production analysis code?
8. How would error handling help when processing many files?
9. Create an example that catches a missing-file error.
10. Explain why debugging and error handling are different skills.

2. Python Data Structures

Lists

1. Create a list containing five sales values.
2. Access the first and last item of a list.
3. Append a new sales value to a list.
4. Remove an item from a list.
5. Sort a list of sales values.
6. Find the length of a list.
7. Slice a list to obtain its first three values.
8. Calculate the total and average of a numeric list.
9. Explain why lists are useful before learning Pandas.
10. Create a list of dictionaries representing three customers.

Tuples

1. Create a tuple containing three values.
2. How is a tuple different from a list?
3. Access the second item in a tuple.
4. Why might an immutable structure be useful?
5. Unpack a tuple into three variables.
6. Can you append to a tuple? Explain.
7. Convert a list into a tuple.
8. Convert a tuple into a list.
9. Create a tuple representing a product record.
10. Give one situation where you would prefer a tuple over a list.

Dictionaries

1. Create a dictionary representing one customer.
2. Access a value using a dictionary key.
3. Add a new key-value pair.
4. Update an existing value.
5. Check whether a key exists.
6. Loop through dictionary keys and values.
7. Create a list containing dictionaries for five customers.
8. Extract all customer names from a list of dictionaries.
9. Why are dictionaries useful for representing structured records?
10. Explain the difference between a dictionary key and a dictionary value.

Sets

1. Create a set of customer IDs.
2. How does a set handle duplicate values?
3. Find the union of two sets.
4. Find the intersection of two sets.
5. Find values present in one set but not another.
6. Add an item to a set.
7. Remove an item from a set.
8. Convert a list with duplicates into a unique collection.
9. Why can sets be useful for checking unique categories?
10. Give a practical data-analysis use case for set operations.

Comprehensions

1. Create a list of squares using a list comprehension.
2. Create a list containing only sales above 1,000.
3. Convert a list of names to uppercase using a comprehension.
4. Create a dictionary comprehension mapping numbers to their squares.
5. Add an `if` condition to a list comprehension.
6. Explain what a comprehension replaces conceptually.
7. When can a comprehension make code harder to read?
8. Create a list of even numbers from 1 to 20.
9. Create a cleaned list of names with whitespace removed.
10. Why should readability be considered when using comprehensions?

3. NumPy

Arrays

1. Import NumPy using its standard alias.
2. Create a one-dimensional NumPy array.
3. Create an array containing numbers 1 through 10.
4. What is a NumPy array and why is it useful for numerical work?
5. Create a two-dimensional array.
6. Check the shape of an array.
7. Check the number of dimensions of an array.
8. Access a specific element of a two-dimensional array.
9. Slice a NumPy array.
10. Explain one advantage of NumPy arrays over ordinary Python lists for numerical computation.

Indexing & Slicing

1. Access the first element of a NumPy array.
2. Access the last element using negative indexing.
3. Select the first three elements.
4. Select every second element.
5. Access a specific row from a two-dimensional array.
6. Access a specific column from a two-dimensional array.
7. Select a rectangular slice from a matrix.
8. Explain what zero-based indexing means.
9. Use Boolean indexing to select values greater than 50.
10. Why is slicing useful when analyzing numerical data?

Mathematical Operations

1. Add 10 to every value in a NumPy array.
2. Multiply every array value by 2.
3. Square every value in an array.
4. Calculate the square root of an array.
5. Add two arrays element by element.
6. Calculate the difference between two arrays.
7. Explain NumPy vectorization.
8. Why is vectorized computation often faster than Python loops?
9. Calculate percentage changes between two arrays.
10. Create an array of prices and apply a 15% discount.

Aggregations

1. Calculate the sum of a NumPy array.
2. Calculate its mean.
3. Find the minimum and maximum.
4. Calculate the standard deviation.
5. Calculate the median.
6. Calculate the sum across rows of a matrix.
7. Calculate the mean across columns.
8. Explain the role of the `axis` argument.
9. Identify the highest sales value in an array.
10. Explain why aggregation functions are important in data analysis.

4. Pandas

Series & DataFrames

1. Import Pandas using its standard alias.
2. Create a Pandas Series from a list.
3. Create a DataFrame from a dictionary.
4. Explain the difference between a Series and a DataFrame.
5. Inspect the first five rows of a DataFrame.
6. Inspect the last five rows.
7. Check the DataFrame's shape.
8. Check its column names.
9. Check its data types.
10. Explain why DataFrames are central to Python-based data analysis.

Reading CSV / Excel

1. Read a CSV file into a DataFrame.
2. Write a DataFrame to a CSV file.
3. Read an Excel worksheet into Pandas.
4. How would you inspect a dataset immediately after loading it?
5. What can go wrong when a CSV uses the wrong delimiter?
6. How can you specify which Excel sheet to read?
7. Why should you inspect column data types after loading a file?
8. How would you handle a file that is too large for memory?
9. Explain the importance of file paths in data-analysis scripts.
10. Create a short loading workflow: read, inspect, validate.

Selecting & Filtering

1. Select one column from a DataFrame.
2. Select multiple columns.
3. Filter rows where sales exceed 10,000.
4. Filter rows using two conditions.
5. Use `isin()` to filter several categories.
6. Filter rows where a text column contains a word.
7. Use `loc` for labeled selection.
8. Use `iloc` for positional selection.
9. Why should parentheses be used around multiple Pandas conditions?
10. Create a filter for high-value customers from a sample DataFrame.

Sorting

1. Sort a DataFrame by sales ascending.
2. Sort by sales descending.
3. Sort by two columns.
4. Reset the index after sorting.
5. Explain the difference between sorting values and sorting indexes.
6. Sort customers by region and then sales.
7. Find the top five sales records after sorting.
8. Find the bottom five records.
9. Why is sorting useful during exploratory analysis?
10. Create a ranked view of products by revenue.

groupby

1. Group sales by region and calculate the total.
2. Group sales by product and calculate the average.
3. Count customers by region.
4. Calculate multiple aggregations in one groupby operation.
5. Explain split-apply-combine.
6. Group by two columns.
7. Find the highest-revenue region.
8. Find the average order value by customer segment.
9. Why is groupby one of the most important Pandas operations for analysts?
10. Recreate a SQL GROUP BY concept using Pandas.

merge / joins

1. Merge two DataFrames using a common customer ID.
2. Explain the difference between inner and left joins.
3. Create a left join between customers and orders.
4. What happens to unmatched rows in a left join?
5. When would an inner join be dangerous for analysis?
6. Merge DataFrames using differently named key columns.
7. Identify duplicate keys before merging.
8. Explain how a many-to-many merge can unexpectedly increase row counts.
9. Validate the row count before and after a merge.
10. Recreate a simple SQL JOIN using Pandas.

concat

1. Concatenate two DataFrames vertically.
2. Concatenate two DataFrames horizontally.
3. Explain the difference between `concat` and `merge`.
4. Combine monthly sales DataFrames into one dataset.
5. Handle mismatched columns during concatenation.
6. Explain what happens to indexes when concatenating.
7. Reset the index after concatenation.
8. Concatenate three regional datasets.
9. Why is concat useful for combining files from repeated periods?
10. Describe a workflow for combining 12 monthly CSV files.

Missing Values

1. Detect missing values in a DataFrame.
2. Count missing values by column.
3. Calculate the percentage of missing values.
4. Drop rows with missing values.
5. Fill missing numeric values with the mean.
6. Fill missing categorical values with a label such as `Unknown`.
7. Explain why blindly filling missing values can be dangerous.
8. Identify columns with excessive missingness.
9. Choose between dropping and imputing missing values in a customer dataset.
10. Explain how missingness can affect business conclusions.

Duplicates

1. Detect duplicate rows.
2. Count duplicate rows.
3. Remove duplicate rows.
4. Remove duplicates based on a specific column.
5. Keep the first occurrence of a duplicate.
6. Keep the last occurrence.
7. Why should duplicates be investigated before deletion?
8. Find duplicate customer IDs.
9. Explain how duplicates can inflate revenue.
10. Create a validation step to check for unexpected duplicates.

Data Types

1. Inspect DataFrame data types.
2. Convert a numeric string column into a numeric type.
3. Convert a text column to datetime.
4. Convert a category column to categorical data.
5. Explain why numeric columns stored as strings are problematic.
6. Handle invalid numeric values during conversion.
7. Extract year from a datetime column.
8. Extract month from a datetime column.
9. Check whether a date column contains invalid values.
10. Why is correct typing important for calculations and visualization?

Dates & Times

1. Create a Pandas datetime column.
2. Extract year, month, and day.
3. Calculate the difference between two dates.
4. Filter records from a particular month.
5. Group sales by month.
6. Create a daily sales summary.
7. Calculate the number of days between order and delivery.
8. Identify orders delivered late.
9. Explain why time-based grouping is important in analytics.
10. Create a monthly revenue trend from transaction dates.

Calculated Columns

1. Create a revenue column from quantity and price.
2. Create a profit column from revenue and cost.
3. Create a margin percentage column.
4. Create a Boolean column for high-value transactions.
5. Use `assign()` to create a calculated column.
6. Use vectorized operations instead of a loop.
7. Handle division by zero in a margin calculation.
8. Create a customer age band column.
9. Explain why calculated columns are useful in EDA.
10. Create three business metrics from an order dataset.

5. Data Visualization

Matplotlib

1. Import Matplotlib's plotting interface.
2. Create a basic line chart.
3. Create a basic bar chart.
4. Add a chart title.
5. Add x- and y-axis labels.
6. Change the figure size.
7. Add a legend.
8. Display grid lines when appropriate.
9. Save a chart to a file.
10. Explain why labeling and readability matter in analyst charts.

Seaborn

1. Import Seaborn.
2. Create a bar chart from a DataFrame.
3. Create a histogram.
4. Create a scatter plot.
5. Create a box plot.
6. Use a categorical variable in a visualization.
7. Add a title and axis labels.
8. Explain when Seaborn is preferable to raw Matplotlib.
9. Create a chart showing sales by region.
10. Create a visualization showing the relationship between price and quantity.

Chart Selection

1. When should you use a line chart?
2. When is a bar chart better than a pie chart?
3. When should you use a histogram?
4. When is a scatter plot appropriate?
5. When should you use a box plot?
6. Choose a chart for monthly revenue over two years.
7. Choose a chart for comparing sales across 10 regions.
8. Choose a chart for analyzing the distribution of order values.
9. Choose a chart for detecting a relationship between ad spend and revenue.
10. Explain why the wrong chart can lead to misleading conclusions.

Data Storytelling

1. What makes a data visualization easy to understand?
2. How should you choose what to emphasize in a chart?
3. Why should unnecessary chart decoration be avoided?
4. How can annotations help communicate an insight?
5. How would you visualize a sudden sales decline?
6. How would you present a KPI to an executive?
7. Why should charts have descriptive titles?
8. How can color be used carefully to highlight a key category?
9. What is the difference between showing data and communicating an insight?
10. Create a short narrative around a hypothetical revenue trend.

6. Exploratory Data Analysis (EDA)

Dataset Understanding

1. What questions should you ask before analyzing a new dataset?
2. How do you inspect the shape of a DataFrame?
3. How do you inspect column types?
4. How do you identify candidate target or KPI columns?
5. Why should you understand the business context before analysis?
6. What is the difference between a row and an observation?
7. How would you identify the grain of a dataset?
8. Why is the dataset's grain important before aggregation?
9. Create a checklist for the first 10 minutes with a new dataset.
10. Explain how understanding the dataset prevents incorrect conclusions.

Outliers

1. What is an outlier?
2. Why can outliers be legitimate business observations?
3. Identify an outlier using a box plot.
4. What is the IQR method?
5. Calculate the IQR from Q1 and Q3.
6. Explain how extreme transaction values can distort an average.
7. When might median be preferable to mean?
8. Should every outlier be removed? Explain.
9. Find potential outliers in order values.
10. Describe a responsible workflow for investigating outliers.

Relationships & Correlation

1. What does correlation measure?
2. What does a positive correlation mean?
3. What does a negative correlation mean?
4. What does a correlation near zero suggest?
5. Why does correlation not prove causation?
6. Calculate a correlation matrix in Pandas.
7. Visualize the relationship between two numeric variables.
8. Which chart is useful for examining two numeric variables?
9. Give an example where two variables might correlate without one causing the other.
10. Explain how correlation can help prioritize further analysis.

EDA Insights

1. How do you turn an observed pattern into a business insight?
2. What is the difference between an observation and a recommendation?
3. How would you investigate a region with unusually low sales?
4. How would you investigate a sudden increase in returns?
5. How would you identify the best-performing customer segment?
6. How can segmentation reveal patterns hidden in overall averages?
7. Why should analysts validate surprising findings?
8. How can SQL, Python, Excel, and Power BI complement each other during EDA?
9. Write three questions you could ask of an e-commerce dataset.
10. Describe the structure of a good EDA conclusion.

7. Real Data Analyst Projects

Sales Analysis Project

1. Define the business objective for a sales-analysis project.
2. What KPIs would you calculate for a sales dataset?
3. How would you calculate revenue and profit?
4. How would you analyze monthly sales trends?
5. How would you identify the best-performing products?
6. How would you compare regional performance?
7. How would you investigate a sales decline?
8. What visualizations would you include in a sales report?
9. What business recommendations could follow from the analysis?
10. Describe an end-to-end workflow from raw sales data to final insight.

Customer Analysis Project

1. Define customer-level KPIs.
2. How would you calculate average order value?
3. How would you identify high-value customers?
4. How would you segment customers?
5. How would you analyze repeat purchases?
6. How would you identify inactive customers?
7. What data-quality problems might appear in customer records?
8. Which charts could show customer behavior clearly?
9. What questions would you ask before recommending a retention strategy?
10. Describe an end-to-end customer-analysis project.

E-commerce Analysis Project

1. What metrics would you track for an e-commerce business?
2. How would you analyze conversion-related data?
3. How would you calculate average order value?
4. How would you analyze product performance?
5. How would you investigate cart abandonment?
6. How would you compare acquisition channels?
7. How would you analyze returns?
8. What dimensions could be used for customer segmentation?
9. Which Python libraries would you use and why?
10. Describe the final deliverables for an e-commerce analytics project.

Marketing Analysis Project

1. What KPIs are commonly used in marketing analytics?
2. How would you compare campaign performance?
3. How would you calculate click-through rate?
4. How would you calculate conversion rate?
5. How would you compare customer acquisition costs?
6. How would you analyze return on ad spend?
7. How would you identify the best-performing channel?
8. How would you visualize campaign trends?
9. What additional data would you request before making a budget recommendation?
10. Describe an end-to-end marketing analytics project.

HR / Employee Analysis Project

1. What KPIs could be used in HR analytics?
2. How would you analyze employee turnover?
3. How would you compare turnover across departments?
4. How would you analyze salary distributions?
5. How could tenure be analyzed?
6. What data-quality issues are common in employee datasets?
7. Which visualizations would help explain workforce composition?
8. How would you investigate a department with unusually high attrition?
9. What privacy considerations should an analyst keep in mind?
10. Describe an end-to-end HR analytics project.

8. Interview Preparation

Python Interview Questions

1. Explain the difference between a list, tuple, set, and dictionary.
2. What is a function and why is it useful?
3. Explain the difference between `==` and `is`.
4. What is list comprehension?
5. What is a DataFrame?
6. How would you handle missing values in Pandas?
7. How would you merge two DataFrames?
8. How would you find duplicate rows?
9. How would you group data and calculate an aggregate?
10. Describe a Python data-analysis project you could explain in an interview.

Pandas Interview Questions

1. How do `loc` and `iloc` differ?
2. How do you filter rows using multiple conditions?
3. What does `groupby()` do?
4. What is the difference between `merge()` and `concat()`?
5. How do you inspect missing values?
6. How do you convert a column to datetime?
7. How do you sort a DataFrame?
8. How do you create a calculated column?
9. How would you identify a many-to-many merge problem?
10. What steps would you take to make a Pandas analysis reproducible?

Case Study Questions

1. A company's revenue fell 15% last month. How would you investigate?
2. A product's sales doubled but profit fell. What could explain it?
3. One region has unusually high revenue. What would you check?
4. Customer churn increased suddenly. What data would you examine?
5. Marketing spend increased but conversions did not. How would you analyze it?
6. A dashboard KPI suddenly changed. What validation steps would you take?
7. Two reports show different revenue numbers. How would you reconcile them?
8. A dataset contains many missing values. What would you do first?
9. A stakeholder asks for a dashboard without defining the business question. What would you ask?
10. How would you communicate an important but uncertain finding to a non-technical stakeholder?